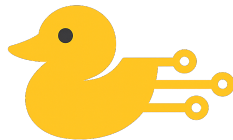


Autonomous Duck Deployment Framework



The Autonomous Duck Deployment Framework

**A Deterministic End-to-End System for
AI-Assisted Software Development**

Framework Version: 1.2 (Flow & Readability
Edition) Core Methodology: Context Engineering
via Local File Sovereignty Operational Model: Dual-
Mode Orchestration (Architect & Builder Modes)

Contents

- 1. Why This Framework Exists
 - 1.1 The Vibe Coding Problem
 - 1.2 The Rubber Duck Principle
 - 1.3 Context Engineering Over Context Windows
 - 1.4 The Central Rule
- 2. The Framework in One Page
- 3. Key Concepts — Read This Before Anything Else
- 4. Your First Session — A Concrete Walkthrough
- 5. A Complete Example — Mini Task Tracker
 - 5.1 Mini Task Tracker DOMAIN.md
 - 5.2 Mini Sprint 001 Acceptance Criteria
- 6. The 8-Step AI Project Deployment Lifecycle
 - Step 1 - Research & Global Architecture
 - Step 2 - Initial Development (The Prototype)
 - Step 3 - Testing (Harness First)
 - Step 4 - Documentation (The Immutable Ledger)
 - Step 5 - Full-Scale Development
 - Step 6 - The Reflection Loop
 - Step 7 - Deployment & Maintenance
 - Step 8 - Resumption (The Pivot Protocol in Practice)
- 7. Operating Modes — Choosing Your Mode
 - 7.1 Mode 1 - Quick Patch
 - 7.2 Mode 2 - Feature Sprint
 - 7.3 Mode 3 - Refactor / Migration
 - 7.4 Mode 4 - Data Ingestion / Discovery
 - 7.5 Mode 5 - Style Audit
 - 7.6 Parallel Sprints
- 8. The Sprint Workflow — The Daily Operating Loop
- 9. The Dual-Mode Orchestration Model
 - 9.1 The File System: The Sovereign Brain
 - 9.2 Builder Permission Levels
 - 9.3 Multi-CLI Protocol and Compute Agnosticism

- 9.4 Token Economics and Cost Management
- 10. Setup and Scaffolding
 - 10.1 System Tooling
 - 10.2 Initializing the Directory
 - 10.3 Full Directory Structure
 - 10.4 Populating Foundational Files
 - 10.5 Context Loading Protocol
 - 10.6 Security and Secrets Protocol
 - 10.7 Git Branching Protocol
- 11. File Templates and Formats
 - 11.1 AGENTS.md
 - 11.2 STATE.md
 - 11.3 DOMAIN.md
 - 11.4 DECISIONS.md (ADR Format)
 - 11.5 COMMANDS.md
 - 11.6 STYLE_GUIDE.md
 - 11.7 SECURITY.md
 - 11.8 GIT_STRATEGY.md
 - 11.9 PROMPT_CHANGELOG.md
 - 11.10 Sprint: requirements.md
 - 11.11 Sprint: blueprint.md
 - 11.12 Sprint: acceptance.md
 - 11.13 Sprint: dry_run.md
 - 11.14 Sprint: implementation_log.md
 - 11.15 Sprint: human_review.md
 - 11.16 Sprint: retrospective.md
 - 11.17 Sprint: rollback_log.md
 - 11.18 planning/backlog.md
- 12. Domain Adaptations and Starter Template Library
 - 12.1 Domain Adaptations
 - 12.2 Starter Template Library
- 13. Prompt Catalog
 - 13.1 Persona Injector
 - 13.2 Project Kickoff Interview Prompt

- 13.3 Resumption Prompt
 - 13.4 Sprint Pack Generation Prompt
 - 13.5 Builder Pre-Flight Dry Run Prompt
 - 13.6 Architect Sanity Audit Prompt
 - 13.7 Builder Implementation Authorization Prompt
 - 13.8 Architect Implementation Review Prompt
 - 13.9 Cross-Sprint Consistency Audit Prompt
 - 14. Quick Reference Checklists
 - 14.1 Before the first AI session
 - 14.2 Before Builder writes code
 - 14.3 Before committing a sprint
 - 15. When Things Go Wrong
 - 16. Glossary
-

1. Why This Framework Exists

Before any instructions, file names, or commands — here is the problem this framework solves and why it is worth your time.

1.1 The Vibe Coding Problem

Vibe coding is what happens when a developer describes an app idea to an AI tool, hits generate, eyeballs the output, and keeps asking for changes in the same chat thread indefinitely. It feels productive because output appears immediately. It fails predictably because of three structural weaknesses that compound over time.

Context bleed and memory bloat

A long chat thread fills with old code blocks, abandoned directions, and conversational filler. The AI loses track of decisions made many messages ago, and regression bugs appear that cannot be explained. You find yourself re-explaining things you already explained three sessions ago.

The improvisation bias

When an AI builder lacks a strict specification, it invents. It creates database keys that do not match your schema, assumes permission levels that clash with your business logic, and takes the path of least resistance — not the path that matches your intent. The AI is not being malicious. It simply fills in gaps with plausible guesses, and those guesses accumulate into a codebase you did not design.

The missing anchor

In vibe coding, conversation history becomes the project memory. That memory is fragile, unsearchable, non-portable, and eventually destroyed by token limits or session loss. When you close the tab, your project context is gone.

1.2 The Rubber Duck Principle

The framework's name comes from rubber duck debugging: explaining code out loud to an inanimate object forces the developer to structure thinking clearly enough to find the problem. This framework applies that principle to AI-assisted development.

Before the AI acts, the human operator structures the problem clearly enough to explain it in writing. The Markdown files are that explanation. The AI only reads what you have written down.

A rubber duck cannot invent your business logic. Neither should your AI.

1.3 Context Engineering Over Context Windows

The common instinct when working with AI is to provide more context — paste the entire codebase, share the full chat history, load everything in. This instinct is counterproductive.

More context is not better context. Cleaner context is better context. Large context windows do not solve the vibe coding problem; they often extend it. Dumping an entire codebase and a long chat history into a prompt is still disorganized context.

Context Engineering is the practice of dividing project rules, history, and specifications into clean, modular Markdown files. Each AI session receives only the files it needs for the current job. The AI reads a precise specification, not a pile of history.

1.4 The Central Rule

The question is never which AI does what. The question is what files must exist before any AI is allowed to act.

If the required documentation does not exist before a session begins, the session does not begin. The AI role is to execute against a specification. Writing the specification is the human operator's job. Chat windows are disposable. The files are permanent.

2. The Framework in One Page

Think of the framework as a small operating system for AI-assisted development. The project brain lives in local Markdown files. The Architect plans the work from those files. The Builder implements only the approved plan. A dry run creates a pause before code changes happen, and every sprint ends by updating the files so the next session starts clean.

The project brain

Local File Sovereignty means the important memory of the project lives in a small set of Markdown files on your computer: `AGENTS.md`, `DOMAIN.md`, `STATE.md`, `DECISIONS.md`, `COMMANDS.md`, and sprint folders. These files survive rate limits, model swaps, lost chats, and long breaks. When you load them into any AI session, you restore the full project context instantly.

The two roles

Architect Mode answers *what* and *why*. It turns raw goals into requirements, blueprints, and acceptance criteria. It never writes source code.

Builder Mode answers *how*. It writes source code only after the blueprint and dry run have been approved by the human operator.

These roles can be played by any capable LLM. What matters is which mode you invoke, not which model you use.

The safety gate

The Dry Run Gate requires the Builder to report exactly what it intends to create or modify before changing any files. That gives the human operator a chance to catch

hidden scope creep, accidental refactors, dependency additions, or business-logic invention — before they happen.

The reset habit

At the end of every sprint, the implementation log, decisions, and current state are written back into the local files. The Builder chat is then closed. The next sprint starts from clean project memory instead of a bloated conversation thread.

Central rule: The question is never which AI does what. The question is what files must exist before any AI is allowed to act.

3. Key Concepts — Read This Before Anything Else

The rest of this document uses specific terms consistently. This section defines the five most important ones so you can read everything that follows without guessing. Full definitions for all terms are in the Glossary (Section 16).

Architect Mode — The planning mode. You invoke an LLM in Architect Mode when you want it to produce requirements, blueprints, and acceptance criteria. It asks clarifying questions, identifies risks, and produces Markdown files. It never writes application source code. Think of it as a senior technical lead who writes the plan but does not touch the keyboard.

Builder Mode — The implementation mode. You invoke an LLM in Builder Mode when you want it to write source code. It reads the sprint blueprint and implements exactly what is specified. It runs a read-only dry run first, stops, waits for your approval, then builds. It never invents business logic that was not in the blueprint.

Sprint Pack — The three files that define a sprint: `requirements.md` (what to build and what not to build), `blueprint.md` (the technical plan, step by step), and `acceptance.md` (the checklist that defines done). The Architect generates these. The Builder consumes them.

Dry Run — A read-only report from the Builder listing every file it intends to create or modify, before it changes anything. The Builder produces the dry run and then stops. You review it, confirm it matches the blueprint, and give permission. Only then does the Builder write code.

Permission Level — A number from 0 to 4 that controls how much the Builder can do. Level 0 is read-only (dry run only). Level 1 is the standard sprint scope. Levels 2–4

allow progressively more freedom for expansions, refactors, and migrations. You assign the level explicitly at the start of each Builder session.

4. Your First Session — A Concrete Walkthrough

This section is a step-by-step script for your very first use of the framework. It uses plain language and shows exactly what to type or paste at each step.

What you will have at the end: a project folder with foundational files, a first sprint pack, a reviewed dry run, and a working starting point that any future AI session can resume from.

Time: approximately 30–45 minutes.

What you need: any capable LLM (Claude, ChatGPT, Gemini), a text or code editor, and a terminal.

Step 1 — Create the project folder

Open your terminal and run:

```
mkdir -p src docs planning/backlog planning/sprints/sprint_001
touch AGENTS.md STATE.md DOMAIN.md DECISIONS.md COMMANDS.md
      STYLE_GUIDE.md SECURITY.md RISKS.md QUESTIONS.md
      FILE_INVENTORY.md
```

This creates the skeleton structure. The files are empty for now. You will fill them in over the next steps.

What these files are for: `DOMAIN.md` holds your business definitions. `STATE.md` holds the current status of the project. `AGENTS.md` tells the AI what rules to follow. `COMMANDS.md` holds the commands to run, test, and build the project. These are the files you paste into every AI session. Full detail on each is in Section 11.

Step 2 — Fill in SECURITY.md first

Before anything else, open `SECURITY.md` and write two lists: what is safe to upload to an AI session, and what must never be uploaded. This file sets the boundary for everything that follows.

A minimal starting version:

```
# SECURITY.md Data Security Protocol

## Never Upload to an AI Session
- API keys or tokens
- Database connection strings or passwords
- Contents of .env files
- Real user names, emails, phone numbers, or IDs

## Safe to Upload
- All framework Markdown files (AGENTS.md, STATE.md, DOMAIN.md, etc.)
- Source code files that contain no secrets
- Sprint planning files
```

Why first? Because every file you create after this may be loaded into an AI session. Setting the boundary before you start means you never accidentally include something you should not.

Step 3 — Fill in AGENTS.md

Copy the template from Section 11.1 into `AGENTS.md`. This file is the rulebook the AI reads at the start of every session. It defines what Architect Mode and Builder Mode are allowed to do.

The template in Section 11.1 is ready to use without modification for your first project. As you learn the framework, you will add project-specific constraints here based on what you learn from each sprint's retrospective.

Step 4 — Write your DOMAIN.md

Open `DOMAIN.md` and describe your project in three parts: what it does, the core entities it works with, and what it explicitly does not handle.

Keep this short and honest. You do not need to have everything figured out. Section 5.1 has a complete filled-in example. Use it as a model.

Why this matters: every rule, entity, or workflow you do not write in `DOMAIN.md` is a gap the AI may fill with its own invention. You control what the AI knows by writing it down.

Step 5 — Run the Project Kickoff Interview

Open a fresh chat window with any capable LLM. Paste the full contents of `AGENTS.md` and your draft `DOMAIN.md`. Then paste this prompt (full version in Section 13.2):

```
Execute Project Kickoff Interview Protocol for [YOUR PROJECT NAME].
```

```
I will provide raw notes, ideas, rough goals, constraints, and known unknowns.
```

```
Your task is to ask up to 10 clarifying questions before creating any final architecture.
```

```
Prioritize questions that affect business logic, data shape, security, user workflows, deployment, and scope boundaries.
```

```
After I answer, generate draft contents for:
```

1. `DOMAIN.md`
2. `STATE.md`
3. `STYLE_GUIDE.md`
4. `docs/architecture.md`
5. `DECISIONS.md` starter ledger
6. `QUESTIONS.md` starter ledger
7. `RISKS.md` starter ledger

```
Do not write application source code. Produce Markdown file blocks only.
```

Answer the AI's questions. Copy the output into the relevant files on your computer. This is Architect Mode in action — the AI is acting as a technical lead helping you define the project before any code is written.

Step 6 — Generate your first Sprint Pack

In the same Architect session, paste your current `STATE.md` and run the Sprint Pack Generation Prompt (Section 13.4):

```
Execute Architect Sprint Generation Protocol.  
  
Active Project State: [paste STATE.md contents]  
Target Sprint Scope: Sprint 001 — [one sentence describing the  
first feature]  
Structural References: Refer to our active DOMAIN.md parameters.  
  
Generate: requirements.md, blueprint.md, acceptance.md.  
Do not output source code.
```

Save the three output files into `planning/sprints/sprint_001/`.

Step 7 — Run the Dry Run

Open a new session in a repository-aware coding tool (Cursor, Claude Code, Cline, or similar). Load the sprint files and paste the Builder Pre-Flight Dry Run Prompt (Section 13.5):

```
Initialize Pre-Flight Dry Run Protocol.  
Permission Level: Level 0 (Read-Only).  
  
Refer to planning/sprints/sprint_001/requirements.md and  
blueprint.md.  
  
Execute a read-only analysis and produce dry_run.md. Then STOP.  
Do not write code until you receive explicit operator clearance.
```

The Builder will produce a `dry_run.md` file listing every file it intends to create or modify. Review it against `blueprint.md`. If it matches, proceed to Step 8. If it lists files outside the blueprint, return to the Architect session and resolve the discrepancy before continuing.

Step 8 — Authorize and build

In the Builder session, type:

```
Dry run approved. Permission Level 1 authorized. Proceed with
implementation
according to requirements.md, blueprint.md, and acceptance.md.

Rules:
1. Modify only approved files.
2. Do not invent business rules not in DOMAIN.md.
3. Log every created or modified file in implementation_log.md.
4. Run all verification commands in acceptance.md and COMMANDS.md.
5. Paste raw terminal output into implementation_log.md.
6. Stop if a blocking ambiguity appears.
```

Step 9 — Verify and close

After the Builder finishes, run the commands in `COMMANDS.md` yourself to verify the work. Work through the five gates in `human_review.md` (Section 11.15). Update `STATE.md` to reflect the completed sprint. Close the Builder session.

Your project is now in a state that any future AI session can resume from by loading `STATE.md`, `DOMAIN.md`, and `AGENTS.md`.

5. A Complete Example — Mini Task Tracker

This section walks a real project through all 8 lifecycle steps. The project is intentionally small so the entire loop is visible from beginning to end.

The project: a simple web app. A user can add tasks, mark them complete, and have their tasks persist across page refreshes. No user accounts. No cloud sync. No collaboration features.

Step 1 — Research & Global Architecture

Before any code is written, you write the business rules. Open `DOMAIN.md` and write:

```

# DOMAIN.md Business Logic and Definitions

## Project Overview
Mini Task Tracker is a local-only task management app for a single user.
It helps a user record small tasks and mark them complete.

## Core Entities

### Task
Definition: A single item the user wants to complete.
Key fields: id, title, status, createdAt, completedAt.
Business rules:
  - A task title cannot be empty. Empty-title submissions must be rejected.
  - A completed task can be reopened (status reverts to "active").
  - Tasks are stored in browser localStorage.
What it is NOT: A shared task, a cloud record, a calendar event, or a reminder.

## Workflows

### Add a Task
1. User types a title and submits.
2. A new Task is created with status "active" and the current timestamp.
3. Task is saved to localStorage and appears in the list immediately.
Edge case: If the title is empty or whitespace-only, reject the submission
           with a visible error message. Do not create the task.

### Complete a Task
1. User clicks a task's complete button.
2. Task status changes to "completed". completedAt is recorded.
3. Task remains visible in the list.
Edge case: Clicking complete on a completed task reopens it
           (status → "active",
            completedAt cleared).

## Out of Scope
- User accounts or authentication
- Multi-device sync
- Email or push notifications
- Team sharing or collaboration
- Cloud storage of any kind

```

You run the Project Kickoff Interview (Section 13.2) with this DOMAIN.md and your raw notes. The Architect asks clarifying questions — framework choice, persistence approach, browser targets — then produces a draft STATE.md, STYLE_GUIDE.md, docs/architecture.md, and starter ledgers for DECISIONS.md, QUESTIONS.md, and RISKS.md.

Step 2 — Initial Development (Sprint 001)

You ask the Architect to generate the Sprint 001 pack.

Sprint goal: build the core task list — add, display, complete, and persist tasks.

The Architect produces:

- `requirements.md` — The user can add a task with a non-empty title. Tasks appear in a list immediately. The user can mark a task complete. Completed tasks can be reopened. Tasks persist after browser refresh. No other features this sprint.
- `blueprint.md` — Three components: `TaskForm`, `TaskList`, and a `localStorage` helper module. Files to create: `src/components/TaskForm.tsx`, `src/components/TaskList.tsx`, `src/utils/storage.ts`.
- `acceptance.md` — See Section 5.2 below.

You review the blueprint and verify each Architect

assumption at the bottom of `blueprint.md`. No blockers.

You create the sprint branch: `git checkout -b sprint/001-core-task-list`.

Step 3 — Testing (Harness First)

Before the Builder writes any implementation code,

`acceptance.md` already defines what done means. The Builder is required to write unit tests before functional code.

Why write tests before code? Because the most common failure in AI-assisted development is building first and checking visually after. If acceptance criteria are not written down, the AI declares success when the UI looks right — even if the underlying logic is broken. The checklist in `acceptance.md` is the actual definition of done, not a visual inspection.

Step 4 — Documentation

After Sprint 001, you record the structural decision in `DECISIONS.md`:

```
## Decision 001: localStorage for persistence
Date: 2026-01-15
Status: Accepted
Type: Data Model

Context: The app needs to persist tasks across browser refreshes
without a backend.

Decision: Use browser localStorage for version 1.

Tradeoffs: localStorage is limited to approximately 5MB and is not
synced across
           devices. This is acceptable because the app is
explicitly single-device
           (see DOMAIN.md Out of Scope). Cloud sync is deferred to
a future version.
```

This entry is permanent. If a future sprint proposes cloud sync, the Architect will find this record and understand the original reasoning.

Step 5 — Full-Scale Development

Sprint 002 adds task filters (All, Active, Completed). Sprint 003 adds the ability to edit a task title. Each sprint follows the same loop: Architect generates the pack, Builder runs the dry run, you approve, Builder implements, you verify, context resets.

Every third sprint, you run the Cross-Sprint Consistency Audit (Section 13.9) to check that the codebase still matches what `DOMAIN.md` says. This catches drift early, before it becomes a larger problem.

Step 6 — Reflection Loop

After Sprint 002, the Builder created a task with an empty title when the user triggered form submission via a keyboard shortcut. This was a gap in the acceptance criteria — the check only covered button clicks.

You record it in `retrospective.md` and add a new constraint to `AGENTS.md`:

```
## Additional Builder Constraints (added Sprint 002 retrospective)
- Always validate non-empty, non-whitespace title before task
  creation,
  regardless of how form submission was triggered.
```

This constraint is now permanent. Every future Builder session reads it.

Step 7 — Deployment & Maintenance

You run `npm run build`. You add the build command and a rollback command to `COMMANDS.md` before pushing to production. Rollback must exist before deployment — not after.

Two weeks later, a user reports a typo in the empty-state message. You fix it as a Mode 1 Quick Patch (Section 7.1) — a single-line change, no sprint pack required, Builder works directly on the dev branch.

Step 8 — Resumption

Three weeks later, you return to the project. You open a fresh Architect session. You load `STATE.md`, `DOMAIN.md`, and `AGENTS.md`. You run the Resumption Prompt (Section 13.3):

```
Execute Resumption Protocol.  
Loaded: STATE.md, DOMAIN.md, AGENTS.md.
```

Summarize:

1. Current project goal
2. Active sprint or paused status
3. Known blockers
4. Last completed step
5. Next safest action
6. Files to load before the next Builder session

The Architect reads the files and tells you exactly where you left off and what to do next. Because the project memory lives in files — not in AI memory — nothing was lost during the three-week break.

5.1 Mini Task Tracker DOMAIN.md

The full completed example is in Step 1 above.

5.2 Mini Sprint 001 Acceptance Criteria

```
# Sprint 001 Acceptance Criteria  
All items must be checked before this sprint is considered  
complete.  
  
## Functional Criteria  
- [ ] User can type a task title and submit it.  
- [ ] An empty or whitespace-only title is rejected with a visible  
error message.  
- [ ] Newly added tasks appear in the list immediately.  
- [ ] User can mark a task complete. Completed state is visually  
distinct.  
- [ ] A completed task can be reopened.  
- [ ] Tasks persist after browser refresh (localStorage).  
  
## Technical Criteria  
- [ ] No TypeScript type errors.  
- [ ] Build completes successfully.  
- [ ] Unit tests written and passing.  
- [ ] Test results pasted into implementation_log.md.  
  
## Verification Commands (run in order)  
1. npm run test  
2. npm run build  
3. npm run lint
```

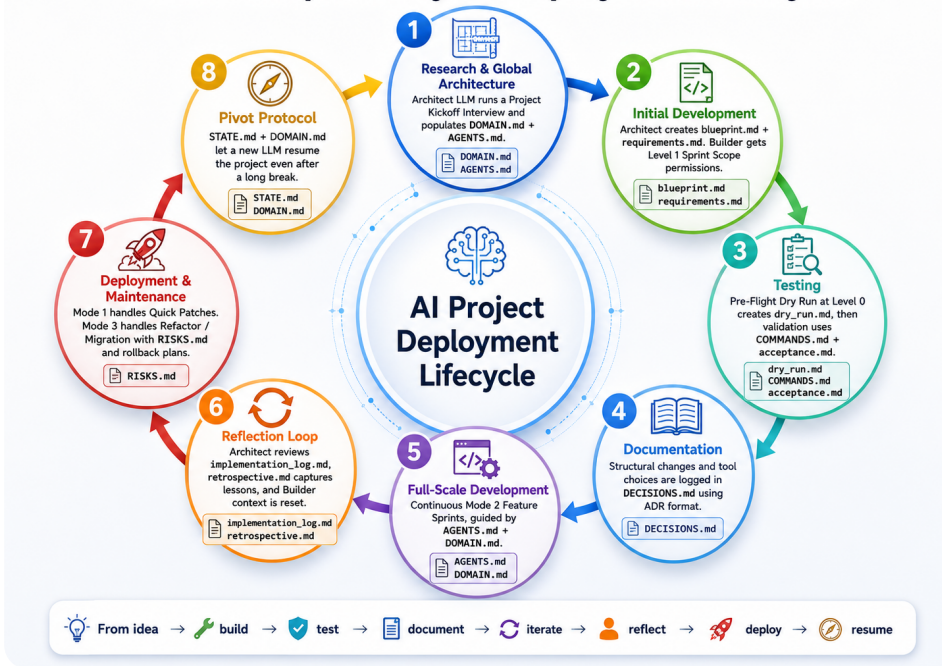
6. The 8-Step AI Project Deployment Lifecycle

The 8-step lifecycle is the central organizing spine of the framework. For a new project, the steps are sequential. After Step 7, loop back to Step 2 for the next feature cycle. Step 8 can happen at any point when returning to a paused project.

How this relates to Operating Modes (Section 7): The lifecycle describes the phases of a project from idea to production. Operating Modes describe how you handle specific tasks within those phases. A project in Phase 5 (Full-Scale Development) typically runs Mode 2 Feature Sprints. A project in Phase 7 (Deployment) might run Mode 1 Quick Patches. The lifecycle is the map. The mode is the vehicle for each journey on that map.

 **[Image: The 8-Step AI Project Deployment Lifecycle — a circular diagram showing the eight steps in sequence: Research & Global Architecture → Initial Development → Testing → Documentation → Full-Scale Development → Reflection Loop → Deployment & Maintenance → Resumption (which feeds back to Initial Development for the next feature cycle). Each node should show the key files it produces or consumes.]**

The 8-Step AI Project Deployment Lifecycle



At a glance, the lifecycle moves from idea to architecture, then prototype, testing, documentation, feature expansion, reflection, deployment, and later resumption.

- **Step 1:** Research & Global Architecture — build the project brain with `DOMAIN.md`, `AGENTS.md`, `STATE.md`, `DECISIONS.md`, `QUESTIONS.md`, `RISKS.md`, and `docs/architecture.md`.
- **Step 2:** Initial Development — create the first working version using `requirements.md` and `blueprint.md`.
- **Step 3:** Testing — define `acceptance.md` and record validation output in `implementation_log.md`.
- **Step 4:** Documentation — update `docs/data_model.md`, `docs/api.md`, and `DECISIONS.md` when structural choices change.
- **Step 5:** Full-Scale Development — run back-to-back Feature Sprints from updated state and sprint folders.
- **Step 6:** Reflection Loop — convert mistakes into constraints through `retrospective.md`, `AGENTS.md`, and `planning/backlog.md`.

- **Step 7:** Deployment & Maintenance — ship safely with `COMMANDS.md`, `RISKS.md`, and `rollback_log.md`.
 - **Step 8:** Resumption — return later by loading `STATE.md`, `DOMAIN.md`, and `AGENTS.md`.
-

Step 1 — Research & Global Architecture

The goal: Build the project brain before writing a single line of code.

Why this phase exists: Every failure mode of AI-assisted development traces back to skipping this step. When the AI has no specification to work from, it invents one. The Step 1 output is the specification that every subsequent phase works from.

The tool: Architect LLM. Claude CLI is preferred when available because it is well-suited for structured planning tasks, but any capable LLM works. The framework is the same regardless of model choice — the files you produce are what matter.

The action: Open the Architect LLM and feed it raw ideas, feature wishlists, client emails, and rough notes. Run the Project Kickoff Interview prompt (Section 13.2). The Architect asks clarifying questions and generates foundational files.

Output files produced: `DOMAIN.md`, `STYLE_GUIDE.md`, `docs/architecture.md`, `STATE.md`, `DECISIONS.md`, `QUESTIONS.md`, `RISKS.md`.

The rule: Do not proceed to Step 2 until `DOMAIN.md` contains real business definitions, `SECURITY.md` is populated, and there are no `QUESTIONS.md` items marked as sprint blockers.

Step 2 — Initial Development (The Prototype)

The goal: Build the first working version of the core feature set under strict Architect control.

The tool: Architect LLM for blueprint generation; Builder agent for execution.

The action: Human reviews ledger files, Architect generates a Sprint Pack, human checks the Blueprint Review Gate, Builder performs a Level 0 Pre-Flight Dry Run, Architect audits, and Builder executes at Level 1 on branch `sprint/001-[description]`.

The rule: The Builder generates only the files listed in the blueprint on the designated branch. No business logic is invented. No files outside the blueprint scope are touched.

Why the Builder cannot invent business logic: If the Builder invents a database key, API endpoint, or permission rule that is not in `DOMAIN.md`, that invention becomes part of the codebase. Future sprints reference it. It compounds. Eventually the AI is maintaining logic it invented that may contradict your actual business rules. The dry run gate exists to catch this before it happens.

Step 3 — Testing (Harness First)

The goal: Establish a testing harness before functional code scales beyond the prototype.

Why this phase matters: The most effective AI development teams invert the vibe coding testing habit. Instead of building first and checking visually after, they build the evaluation framework before the AI writes implementation code. A visual check is not acceptance. A passing test suite is.

The action: Before the Builder writes new functional code for a sprint, mandate that it writes the unit tests first. The `acceptance.md` checklist drives what must be proven before the sprint closes.

The rule: The Builder must run all commands listed in `COMMANDS.md` and paste terminal output directly into `implementation_log.md`. Visual inspection alone is not acceptance.

Step 4 — Documentation (The Immutable Ledger)

The goal: Keep the architectural record current as the codebase grows.

Why this phase matters: Documentation debt is the silent killer of AI-assisted projects. If context files fall out of sync with code, the AI begins improvising again. An Architect that reads a `DOMAIN.md` from three sprints ago will give you a blueprint based on a project that no longer exists.

The action: After every sprint that introduces a structural change, command the Architect to update `docs/data_model.md` and `docs/api.md`.

The rule: Any change to the data model, API contract, auth system, or external dependencies requires a new `DECISIONS.md` entry in ADR format before the next sprint begins.

Step 5 — Full-Scale Development

The goal: Execute the complete product build through back-to-back Feature Sprints.

Why this phase matters: At scale, improvisation bias compounds. A Builder that invents one database key in Sprint 3 may reference it in Sprint 7, Sprint 12, and Sprint

18. The Cross-Sprint Consistency Audit (Section 13.9) exists specifically to catch this drift before it becomes irreversible.

The action: Execute Mode 2 Feature Sprints in sequence. Close the Builder chat completely between every sprint. Every third sprint, run the Cross-Sprint Consistency Audit to catch drift.

The rule: Each new Builder session begins by loading only the freshly updated `STATE.md` and the current sprint folder. The context reset between sprints is non-negotiable.

Why close the Builder chat between sprints? A long Builder chat accumulates context bleed. Old sprint decisions, abandoned approaches, and previous implementation details fill the context window and start influencing the next sprint. Starting fresh means the Builder reads only what you intentionally loaded — no carryover noise.

Step 6 — The Reflection Loop

The goal: Turn one-time errors into permanent constraints, keep the context window clean, and maintain the backlog.

Why this phase matters: AI coding tools repeat mistakes unless explicitly told not to. A Builder that created an empty-title task in Sprint 2 will create another in Sprint 8 unless a constraint in `AGENTS.md` prevents it. The retrospective converts experience into guardrails.

The action: Architect reviews log loops, human fills in retrospective metrics, updates constraints inside `AGENTS.md`, triages backlog items, and closes the Builder thread completely.

Step 7 — Deployment & Maintenance

The goal: Move the product to production and manage it reliably over time.

The action: Minor fixes match Mode 1 Quick Patches. Heavy database migrations match Mode 3 Restructures.

The rule: Rollback commands must be present in `COMMANDS.md` before deployment. Fragile production assumptions are logged in `RISKS.md` with specific mitigation plans. Rollback must be ready before you deploy — not after something breaks.

Step 8 — Resumption (The Pivot Protocol in Practice)

The goal: Return to any paused project after days, weeks, or months in minutes.

Why this phase matters: Because project intelligence lives in Markdown files and not in AI memory, there is nothing to reconstruct. Load the key files and continue. The three-week break in the Mini Task Tracker example (Section 5) costs two minutes to resume — not two hours.

The action: Update `STATE.md` before session close. When resuming, load `STATE.md`, `DOMAIN.md`, and `AGENTS.md` into the Architect LLM and run the Resumption Prompt (Section 13.3).

The rule: `STATE.md` must be updated at the close of every session without exception. This is the single habit that makes everything else work. If you do only one thing consistently, make it this.

7. Operating Modes — Choosing Your Mode

Operating Modes define how to handle individual tasks within the larger lifecycle. Choose your mode before opening any AI session.

How this relates to the 8-Step Lifecycle: The lifecycle tells you which phase of the project you are in. An Operating Mode tells you how to approach the specific task in front of you right now. You can be in Lifecycle Phase 5 (Full-Scale Development) and switch between Mode 1 Quick Patches and Mode 2 Feature Sprints depending on what needs doing. The lifecycle is the map. The mode is the vehicle.

Situation	Mode	Lifecycle Phase
Starting a brand-new project	Phase 1 Kickoff (pre-mode)	Phase 1
Building the first working feature	Mode 2 - Feature Sprint	Phase 2
Adding a new feature	Mode 2 - Feature Sprint	Phase 5
Fixing a bug or small config change	Mode 1 - Quick Patch	Phase 7
Upgrading a dependency or migrating data	Mode 3 - Refactor/ Migration	Phase 7
Parsing raw data files or mapping a new API	Mode 4 - Data Ingestion	Phase 1 or 5
Enforcing code style across new files	Mode 5 - Style Audit	Phase 5 or 7
Running two non-overlapping features at once	Mode 2x2 - Parallel Sprints	Phase 5
Returning to a paused project	Phase 8 Resumption (pre-mode)	Phase 8

7.1 Mode 1 — Quick Patch

Use for: Typo fixes, minor UI adjustments, single-function changes, renaming, or small config edits. Any change under approximately 10 lines that introduces no new business logic.

Workflow / rule: Open Builder at Level 1 permission on dev branch → state the specific file and change → Builder mutates file → run verification from `COMMANDS.md` → commit → update `STATE.md`.

When NOT to use this mode: If the fix requires understanding how multiple files interact, introduces any new logic, or touches a database schema — use Mode 2 or Mode 3. The boundary is: does this change require a blueprint? If yes, it is not a Quick Patch.

7.2 Mode 2 — Feature Sprint

Use for: New features, new routes, new components, new integrations, or any change that touches more than one file or introduces new business logic.

Workflow / rule: Pre-flight checklist — no open blocking `QUESTIONS.md`, no unresolved High/High `RISKS.md`, Architect loaded with `AGENTS.md` + `STATE.md` + `DOMAIN.md`, branch instantiated, sprint folder templates initialized.

7.3 Mode 3 — Refactor / Migration

Use for: Database migrations, major dependency upgrades, architectural restructuring, auth system changes, or anything that could break existing functionality.

Workflow / rule: Builder operates at Level 3 or Level 4, assigned explicitly. Architect reviews `implementation_log.md` before merge to dev or main. Retrospective is mandatory.

Important: `rollback_log.md` must be pre-filled with the current clean commit hash and a rollback command before the Builder begins. If anything goes wrong, you need to be able to revert in seconds, not minutes.

7.4 Mode 4 — Data Ingestion / Discovery

Use for: Parsing messy spreadsheets or PDFs, ingesting external APIs, mapping new data sources, or building ETL pipelines.

Workflow / rule: Security gate first — all PII replaced with synthetic placeholders, API keys replaced with `[REDACTED]`, files reviewed against `SECURITY.md` never-share list before anything is loaded into an AI session.

7.5 Mode 5 — Style Audit

Use for: Enforcing `STYLE_GUIDE.md` conventions across new or modified files.

Workflow / rule: Recommended after any sprint that creates more than three new files. Mandatory before production deployment.

7.6 Parallel Sprints

Use for: Two or more feature sprints running simultaneously.

Workflow / rule: Permitted only when blueprints list zero overlapping files, each has its own branch/folder/Builder session, and the Architect Sanity Audit (Section 13.6) confirms zero file overlap.

Warning: Parallel sprints compound the risk of conflict. A single overlapping file in the two blueprints can cause a merge conflict that invalidates both sprints. Use conservatively.

8. The Sprint Workflow — The Daily Operating Loop

The sprint workflow is the step-by-step process for any task larger than a quick patch. Use it for Mode 2, Mode 3, and Mode 4. Each step names what the human does, not just what the AI produces.

Step 1 — Pre-Flight Gate

What the human does: Check `QUESTIONS.md` for any open items that would prevent a sound blueprint. Check `RISKS.md` for unresolved High severity risks. Confirm the sprint branch has been created in git.

Main artifact: Verified `QUESTIONS.md`, `RISKS.md`, git branch.

Gate question to ask yourself: Is there anything I do not yet know that would cause the Architect to make a wrong assumption? If the answer is yes, resolve it now. Do not generate a blueprint against an unknown.

Step 2 — Prime the Architect

What the human does: Open a fresh Architect session. Load `AGENTS.md`, `STATE.md`, `DOMAIN.md`, and any recent `DECISIONS.md` entries. Run the Sprint Pack Generation Prompt (Section 13.4).

Main artifact: `requirements.md`, `blueprint.md`, `acceptance.md`.

Step 2B — Human Blueprint Review Gate

What the human does: Read `blueprint.md` carefully before authorizing anything. Verify each Architect assumption listed at the bottom of the file. Ask yourself:

- Does this blueprint match what I actually want built?

- Does any assumption conflict with `DOMAIN.md`?
- Are there files listed that should not be touched this sprint?
- Is the scope narrower or broader than intended?

If any assumption is wrong, write your corrections in `blueprint_feedback.md` and return to Step 2 for a revised blueprint. **Do not skip this gate.** A flawed blueprint produces a flawed build. Catching a problem here takes minutes. Catching it after implementation takes hours.

Main artifact: `blueprint_feedback.md` (if corrections needed).

Step 3 — Pre-Flight Dry Run

What the human does: Open a new Builder session. Load the sprint files. Run the Builder Pre-Flight Dry Run Prompt (Section 13.5). Wait for the `dry_run.md` report. The Builder stops after producing it.

Main artifact: `dry_run.md`.

Gate question to ask yourself: Does every file in `dry_run.md` appear in `blueprint.md`? If the Builder intends to touch a file that is not in the blueprint, that is scope creep. Send it back before any code is written.

Step 4 — Architect Sanity Audit

What the human does: Paste `requirements.md`, `blueprint.md`, `acceptance.md`, and `dry_run.md` into the Architect session. Run the Architect Sanity Audit Prompt (Section 13.6). The Architect checks for hidden scope bleed, missing verification commands, and undeclared dependencies.

Main artifact: Permission approval or rejection with required changes.

Step 5 — Dependency Approval Gate

What the human does: If the dry run or audit revealed any new dependencies (npm packages, pip packages, external APIs), create a `DECISIONS.md` entry for each before authorizing the build.

Main artifact: Updated `DECISIONS.md`.

Why before the build? A dependency added without a decision record is a dependency that future AI sessions will not know the reasoning for. If you later need to audit which sprint introduced a vulnerable library, the ADR tells you why it was added and who approved it.

Step 6 — Authorize and Build

What the human does: Type the authorization message in the Builder session (full text in Section 13.7). The Builder implements according to the approved blueprint and logs every change in `implementation_log.md`.

Main artifact: Source files, `implementation_log.md`.

Step 7 — Run Verification

What the human does: The Builder runs all commands listed in `acceptance.md` and `COMMANDS.md` and pastes the raw terminal output into `implementation_log.md`. Read the actual output — not just the summary.

Main artifact: Terminal output in `implementation_log.md`.

Why read the raw output? Because “tests passed” in a summary can mean 1 test passed and 47 were skipped. The raw output tells you what actually ran.

Step 8 — Architect Reviews Output

What the human does: Paste `acceptance.md`, `dry_run.md`, and `implementation_log.md` into the Architect session. Run the Architect Implementation Review Prompt (Section 13.8).

Main artifact: Architect review — approved or list of required fixes.

Step 9 — Human Approval Gate

What the human does: Work through the five gates in `human_review.md`:

1. **Scope:** Does `implementation_log.md` list only files that appeared in `dry_run.md`?
2. **Tests:** Are passing test results present in `implementation_log.md`?
3. **Debt:** Are all known issues from this sprint recorded in `planning/backlog.md`?
4. **Decisions:** Do all new dependencies have `DECISIONS.md` entries?
5. **State:** Does `STATE.md` reflect the completed sprint status?

All five gates must pass before committing.

Step 10 — Commit, Update, Reset

What the human does: Commit the sprint branch. Merge to dev. Update `STATE.md`. Complete `retrospective.md`. Close the Builder thread.

Main artifact: Git commit, updated `STATE.md`, `retrospective.md`.

The reset is not optional. Closing the Builder thread is how the framework prevents context bleed from accumulating across sprints. The next sprint starts from what is in the files — not from what was in the conversation.

9. The Dual-Mode Orchestration Model

The framework is anchored by a Dual-Mode Orchestration Model. Any capable LLM can execute any task if the operator clearly shifts between two operational modes: Architect Mode and Builder Mode.

Architect Mode

Architect Mode can run in any capable LLM, including Claude Pro, ChatGPT Plus, Gemini Advanced, a terminal interface, or a browser project sandbox. Its responsibility is to dictate the *what* and *why*: process definitions, map relational fields, mitigate structural code risk, resolve logical unknowns, and produce development blueprints.

Its outputs are `requirements.md`, `blueprint.md`, and `acceptance.md`. Its strict constraint is that it does not write codebase source files directly, does not read live runtime logs, and explicitly catalogs structural boundaries.

Why the Architect cannot write code: If the Architect writes source code, it is simultaneously deciding what to build and how to build it. That collapses the planning and execution separation that makes this framework work. The Architect sets the standard. The Builder is measured against it. If they are the same session, there is no independent standard.

Builder Mode

Builder Mode runs in a local repository-aware coding environment such as Cursor, Claude Code, Cline, or Codex. Its responsibility is to dictate the *how*: consume approved schematics, generate source files inside `/src`, execute local tests, and register environment updates.

Its outputs are source files, `dry_run.md`, `implementation_log.md`, and test output. Its strict constraint is that it may not invent unapproved business rules. It must execute a read-only dry run before changing files and must stop after the dry run until permission is granted.

Why the Builder must stop after the dry run: The pause between planning and execution is the entire safety model of the framework. If the Builder could skip the dry run, there would be no window to catch scope creep before the code is written. The dry run is the review step.

9.1 The File System: The Sovereign Brain

The core brain of the framework is localized on your drive within an organized collection of Markdown tracking files. Because instructions survive independently of any model state, you can run an Architect session in Claude, hit a rate limit, copy the updated tracking files, drop them into ChatGPT or Gemini, and pick up without context degradation.

This is why Local File Sovereignty is the foundational principle. The files are the project. The AI is the tool.

9.2 Builder Permission Levels

Level	Name	Operational Execution Parameters
0	Read-Only	Structural analysis and file scanning only. Zero code mutation or file creation permitted. Used for pre-flight dry runs.
1	Sprint Scope	Standard starting default. The agent can modify or create only files explicitly cataloged in the active sprint blueprint block.
2	Approved Expansion	The agent can generate minor auxiliary helper modules if surfaced during the dry run and validated by the Architect pack.
3	Supervised Refactor	Authorized to mutate adjacent files outside the core blueprint path, provided it indexes every change inside logging modules.
4	Migration	Heavy schema adjustments or complete architecture overhauls. Requires explicit rollback logs before processing begins.

Start at Level 1 for almost every sprint. Level 2 is appropriate when the dry run reveals a helper module the blueprint did not anticipate but the Architect approves. Levels 3 and 4 are for deliberate, high-stakes changes where you want a maximum audit trail.

9.3 Multi-CLI Protocol and Compute Agnosticism

Models are treated as interchangeable, temporary compute engines piped into local shell utilities. The developer commands the ecosystem by loading system configuration files into terminal arguments.

The framework is compute-agnostic: models are temporary engines that read the same local files.

- **Claude CLI** is well-suited for schema planning, blueprint assembly, and code-review checks. Its structured reasoning makes it effective for Architect tasks. Example: `cat AGENTS.md STATE.md DOMAIN.md | claude "Compile Sprint Pack NNN"`
- **Gemini CLI** is well-suited for high-capacity ingestion during data parsing discovery gates due to its large context window. Example: `cat messy_log.csv | gemini "Map data vectors per AGENTS.md rules"`
- **GPT CLI** is well-suited for structural code style verification and rule compliance checks. Example: `cat STYLE_GUIDE.md src/app.ts | gpt "Identify design standard fractures"`

These are recommendations, not requirements. The framework works with any combination of models. What matters is that the same local files feed every model — that consistency is what makes models interchangeable.

9.4 Token Economics and Cost Management

Keep runtime expenses efficient by parsing compact data payloads, splitting large inventory lists, and routing basic text translation tasks to smaller, highly optimized models such as GPT-4o-mini or Claude Haiku.

10. Setup and Scaffolding

10.1 System Tooling

Use simple tools at the start. Add terminal and CLI layers only when they actually improve the workflow.

- **Obsidian** — primary interface for managing Markdown files, Architect Packs, `STATE.md`, prompts, and the project brain. Its linked notes and search make it easy to navigate between files.
- **VSCode** — primary interface for the Builder layer; AI coding extensions execute from here.
- **WSL on Windows** — Bash environment for scripts, git commands, and local servers.
- **Git** — every sprint ends with a clean commit on its branch before context reset.
- **Claude CLI** — Architect sessions via terminal.
- **Gemini CLI** — discovery and data ingestion sessions.
- **GPT CLI** — validation and style enforcement sessions.

You do not need all of these on day one. Start with a text editor and any LLM chat interface. Add terminal tools and CLIs when you feel the friction they solve.

10.2 Initializing the Directory

```
# Create primary project directory
mkdir -p /mnt/c/Users/Public/Projects/[PROJECT_NAME]
cd /mnt/c/Users/Public/Projects/[PROJECT_NAME]

# Initialize git and create branch structure
git init
git checkout -b main
git checkout -b dev

# Build structural scaffolding
mkdir -p src docs planning/meetings planning/sprints/sprint_001
      _templates

# Generate root tracking files
touch AGENTS.md README.md STATE.md DECISIONS.md DOMAIN.md RISKS.md
      QUESTIONS.md FILE_INVENTORY.md COMMANDS.md STYLE_GUIDE.md
      SECURITY.md GIT_STRATEGY.md PROMPT_CHANGELOG.md

# Generate documentation files
touch docs/architecture.md docs/data_model.md docs/api.md docs/
      permissions.md docs/validation.md

# Generate sprint templates
touch planning/backlog.md
touch planning/sprints/sprint_001/requirements.md planning/
      sprints/sprint_001/blueprint.md planning/sprints/
      sprint_001/blueprint_feedback.md planning/sprints/
      sprint_001/acceptance.md planning/sprints/sprint_001/
      dry_run.md planning/sprints/sprint_001/
      implementation_log.md planning/sprints/sprint_001/
      human_review.md planning/sprints/sprint_001/
      retrospective.md planning/sprints/sprint_001/
      rollback_log.md

# Create .gitignore
echo -e ".env\n.env.*\n*.key\n*.pem\nsecrets/" > .gitignore
```

What this creates: The root tracking files are the project brain. The `docs/` folder holds architecture and API documentation. The `planning/` folder holds the backlog and sprint folders. The `src/` folder is where the Builder writes code. The `_templates/` folder holds starter files for new sprints.

10.3 Full Directory Structure

```
Project-Root/
├── AGENTS.md           # AI persona rules and permission
levels
├── STATE.md           # Living roadmap: current sprint,
status, blockers
├── DOMAIN.md          # Business logic, entities, and
workflows
├── DECISIONS.md       # Immutable ADR ledger of design
choices
├── FILE_INVENTORY.md  # Source asset manifest and data shapes
├── COMMANDS.md        # Run, build, test, deploy, and
rollback commands
├── STYLE_GUIDE.md     # Naming conventions, UI rules, code
standards
├── SECURITY.md         # Safe-to-share and never-share file
lists
├── GIT_STRATEGY.md    # Branching model and commit rules
├── PROMPT_CHANGELOG.md # Version history of all framework
prompts
├── RISKS.md           # Known risks and mitigation strategies
├── QUESTIONS.md        # Unresolved questions blocking design
├── README.md          # Project overview
├── .gitignore         # Excludes secrets and env files
├── docs/
│   ├── architecture.md, data_model.md, api.md, permissions.md,
validation.md
├── planning/
│   ├── backlog.md     # Future ideas outside current scope
│   └── sprints/
│       └── sprint_001/
│           ├── requirements.md, blueprint.md, acceptance.md
│           ├── blueprint_feedback.md
│           ├── dry_run.md
│           ├── implementation_log.md
│           ├── human_review.md
│           ├── rollback_log.md
│           └── retrospective.md
├── _templates/        # Starter templates for new sprints
└── src/               # Execution codebase
```

10.4 Populating Foundational Files

Fill files in this order. The order matters — each file builds on the previous.

1. **SECURITY.md** — Fill in safe-to-share and never-share lists before anything else. Every file you create after this may be uploaded to an AI session. Know the boundary first.

2. **AGENTS.md** — Paste the permission rules and persona instructions. This file is loaded in every session and is the highest-priority instruction set.
3. **DOMAIN.md** — Write business definitions. Every entity, workflow, and rule not defined here is a gap the AI may fill with an invention. The Mini Task Tracker example in Section 5 shows what a completed `DOMAIN.md` looks like.
4. **COMMANDS.md** — Document every command needed to run, test, build, deploy, and roll back before writing code. The Builder will need these to run verification.
5. **GIT_STRATEGY.md** — Fill in the branching model for this project.
6. **STATE.md** — Initialize using the strict short format. Set status to Discovery Phase. Example below.
7. **DECISIONS.md** — Create an empty ledger with the ADR header ready.
8. **QUESTIONS.md and RISKS.md** — Create empty ledgers with headers. Add any known risks or questions immediately.

Example initialized STATE.md:

```
# STATE.md

## Current Goal
Build the Mini Task Tracker – a local task management web app for
a single user.

## Active Sprint
None yet – in Discovery Phase.

## Current Status
Discovery

## Known Blockers
None

## Last Completed Step
Initialized project directory and populated foundational files.

## Next Intended Step
Run Project Kickoff Interview to generate DOMAIN.md,
STYLE_GUIDE.md, docs/architecture.md.

## Files Recently Changed
AGENTS.md - populated with permission rules
SECURITY.md - populated with safe/never-share lists
DOMAIN.md - first draft written
```

10.5 Context Loading Protocol

Loading the wrong files wastes tokens and introduces noise. Load only the files required for the session type.

Session Type	Load These Files	Do Not Load	Security Check
Architect Planning	AGENTS.md, STATE.md, DOMAIN.md, DECISIONS.md, QUESTIONS.md, RISKS.md	Full source code, old sprint folders	No - these files should never contain secrets
Builder Implementation	AGENTS.md, STATE.md, sprint requirements + blueprint + acceptance, COMMANDS.md, and only source files the sprint touches	DECISIONS.md, old sprints, unrelated files	No - but verify COMMANDS.md has no inline credentials
Audit Verification	acceptance.md, dry_run.md, implementation_log.md, test output	Source files, DOMAIN.md	No
Discovery Data Ingestion	AGENTS.md, FILE_INVENTORY.md, raw source documents	Sprint files, source code	Yes - redact all PII and credentials before loading
Validation Style Check	AGENTS.md, STYLE_GUIDE.md, files being validated	DOMAIN.md, sprint files	No

10.6 Security and Secrets Protocol

Highest-priority rule: API keys, database connection strings, OAuth secrets, private keys, .env contents, real user PII, internal IPs/ private domains, and payment information must never appear in files uploaded to an LLM session.

- **Redaction rule for Mode 4 Discovery:** replace sensitive values with REDACTED or synthetic placeholders before the file leaves your machine.
- **Safe-to-share list:** framework Markdown files are safe to upload only if written according to the protocol and free of secrets.

- The `.gitignore` file must include `.env`, `.env.*`, `*.key`, `*.pem`, and `secrets/` directories before the first commit.
- `SECURITY.md` is the project-specific record of what is safe and not safe for this project. Update it when new credential types are introduced.

10.7 Git Branching Protocol

Branch	Purpose	Who Commits
main	Production-ready code only.	Human operator, after Architect approval.
dev	Integration branch. All sprint branches merge here first.	Human operator, after <code>human_review.md</code> passes.
sprint/NNN-short-description	One branch per sprint.	Builder agent.

- The Builder always works on a named sprint/NNN branch. This is stated explicitly at the start of every Builder session.
 - Mode 1 Quick Patch is the only mode that may work directly on dev, given its scope constraint of under 10 lines with no new business logic.
 - Mode 3 Refactor/Migration requires a dedicated `refactor/description` branch with an explicit merge plan in `blueprint.md`.
-

11. File Templates and Formats

Copy these layouts into local Markdown files. The templates are intentionally simple so they are easy to maintain and easy for an AI session to parse.

How to use these templates: Copy the template, fill in the placeholders, and save the file to your project root or sprint folder. You do not need to fill every field on day one. `DECISIONS.md` and `QUESTIONS.md` start empty. `DOMAIN.md` and `STATE.md` should be populated before the first Architect session.

Where to find completed examples: `DOMAIN.md` — Section 5, Step 1. `acceptance.md` — Section 5.2. `DECISIONS.md` — Section 5, Step 4. `STATE.md` — Section 10.4.

11.1 AGENTS.md

```
# AGENTS.md System Operational Rules

## Architect Mode Constraints
When invoked in Architect Mode, you operate strictly at the
systems specification layer.
1. Never write application source files or code blocks directly.
2. Enforce a structural Discovery Gate before approving
implementation work.
    Interrogate code constraints and edge cases.
3. Your primary outputs are structured Markdown blueprints:
    requirements.md, blueprint.md, and acceptance.md.

## Builder Mode Constraints
When invoked in Builder Mode, you act as a localized
implementation contractor.
1. Never invent database keys, entity behaviors, or business rules
omitted from DOMAIN.md.
2. Operate strictly within the permission level allocated by the
human operator
    for the active session.
3. You must execute an automated Read-Only pre-flight dry run and
stop generation
    immediately to await human approval.

## Builder Permission Levels
- Level 0: Read-only. Structural scanning and dry run execution
only.
    No code mutation permitted.
- Level 1: Standard Sprint Scope. Mutate only files explicitly
logged
    in the active blueprint sheet.
- Level 2: Approved Expansion. Allowed to build secondary utility
scripts
    highlighted in the dry run.
- Level 3: Supervised Refactor. Authorized to clean adjacent
files,
    provided all edits are recorded in logs.
- Level 4: Migration. Heavy structural schema updates.
    Requires pre-written rollback procedures.
```

11.2 STATE.md

```
# STATE.md

## Current Goal
[One sentence: what are we building right now?]

## Active Sprint
Sprint [NUMBER]: [Brief description]
Branch: sprint/[NNN]-[description]

## Active Parallel Sprint (if any)
Sprint [NUMBER]: [Brief description]
Overlap check: [date confirmed]

## Current Status
[Discovery / Planning / In Progress / Verification / Complete /
Blocked]

## Known Blockers
[Blocker or "None"]

## Last Completed Step
[What was the last thing finished?]

## Next Intended Step
[What is the very next action?]

## Files Recently Changed
[filename] - [brief note on what changed]
```

A completed example is in Section 10.4.

11.3 DOMAIN.md

```
# DOMAIN.md Business Logic and Definitions

## Project Overview
[Two to three sentences describing what this project does and why
it exists.]

## Core Entities

### [Entity Name]
Definition: [Exact meaning in this project]
Key fields: [Important properties]
Business rules: [How this entity behaves]
What it is NOT: [Clarify common confusions]

## Workflows

### [Workflow Name]
1. [Step 1]
2. [Step 2]
Edge case: [What happens if X fails?]

## Out of Scope
This project explicitly does NOT handle:
[Item 1]
[Item 2]
```

A fully completed example is in Section 5, Step 1.

11.4 DECISIONS.md (ADR Format)

```
# DECISIONS.md Architecture Decision Ledger
Decisions are never deleted - only superseded by new entries.

## Decision 001: [Short title]
**Date:** [YYYY-MM-DD]
**Status:** [Accepted / Superseded by Decision NUMBER]
**Type:** Architecture / Dependency / Data Model / Security /
Other

### Context
[What situation forced this decision?]

### Decision
[What was decided?]

### Tradeoffs
[What this costs or complicates]
```

A completed example is in Section 5, Step 4.

11.5 COMMANDS.md

```
# COMMANDS.md Project Commands

## Environments
| Environment | Base URL | Database | Notes |
|---|---|---|---|
| dev | localhost:[port] | [local DB] | Safe to reset |
| staging | [staging URL] | [staging DB] | QA only |
| production | [prod URL] | [prod DB] | Requires approval |

## Development [dev only]
[command to start dev server]

## Testing [dev and staging]
[command to run all tests]

## Build
[command to build for production]

## Lint / Type Check
[lint command]

## Rollback
[rollback command – must be documented before deployment]
```

11.6 STYLE_GUIDE.md

```
# STYLE_GUIDE.md Project Conventions

## Code Style
Language: [e.g. TypeScript strict mode]
Formatter: [e.g. Prettier]
Linter: [e.g. ESLint]

## Naming Conventions
| Type | Convention | Example |
|---|---|---|
| Components | PascalCase | UserProfileCard |
| Functions | camelCase | getUserById |
| Constants | SCREAMING_SNAKE | MAX_RETRY_COUNT |

## What Requires Architect Approval Before Proceeding
- New database tables or schema changes
- New third-party dependencies
- Auth or permission changes
```

11.7 SECURITY.md

```
# SECURITY.md Data Security Protocol

## The Absolute Rule
The following must NEVER appear in any file uploaded to an LLM
session:
- API keys or tokens of any kind
- Database connection strings or passwords
- Contents of env or .env.* files
- Real user PII: names, emails, phone numbers, addresses, ID
  numbers

## Redaction Protocol for Mode 4 Discovery
Before uploading any operational data file:
1. Replace all email addresses with user_NNN@example.com
2. Replace all API keys with [API_KEY_REDACTED]
3. Save the redacted version as [filename]_redacted.[ext]

## Safe to Upload for This Project
[List the specific file types and folders safe to share for this
project]

## Never Upload for This Project
[List specific file types and folders that must never be shared
for this project]
```

11.8 GIT_STRATEGY.md

```
# GIT_STRATEGY.md

## Branch Model
- main: production-ready only
- dev: integration branch
- sprint/NNN-short-description: one branch per sprint
- refactor/short-description: used for Mode 3 refactor or
  migration work

## Rules
1. Builder works only on the assigned branch.
2. Sprint branches merge into dev only after human_review.md
  passes.
3. dev merges into main only after Architect approval.
4. Mode 1 quick patches may work directly on dev only when under
  10 lines
   and no business logic is introduced.
```

11.9 PROMPT_CHANGELOG.md

```
# PROMPT_CHANGELOG.md

## Prompt Version Log

### Version [NUMBER] - [YYYY-MM-DD]
**Prompt affected:** [Prompt name]
**Change type:** Added / Revised / Deprecated
**Reason:** [Why the change was made]
**Impact:** [How future sessions should behave differently]
```

11.10 Sprint: requirements.md

```
# Sprint [NUMBER] Requirements
**Sprint Goal:** [One sentence]
**Status:** Planning / In Progress / Complete
**Branch:** sprint/[NNN]-[description]

## Functional Requirements
1. [Requirement 1]

## Scope Boundaries

### In Scope
- [Feature A]

### Out of Scope (do not build this sprint)
- [Feature X]
```

11.11 Sprint: blueprint.md

```
# Sprint [NUMBER] Technical Blueprint
**Architect:** [LLM used]

## Implementation Plan

### Step 1: [Step Name]
What: [Description]
Files affected: [list]
Logic: [How it works]

## Architect Assumptions
- [List every assumption made. Human must verify each before approving.]
```

11.12 Sprint: acceptance.md

```
# Sprint [NUMBER] Acceptance Criteria
All items must be checked before this sprint is considered
complete.

## Functional Criteria
- [ ] [User can do X]
- [ ] [Feature Y works correctly]

## Technical Criteria
- [ ] No type errors
- [ ] Build completes successfully
- [ ] Test results pasted into implementation_log.md

## Verification Commands (run in order)
1. [test command]
2. [build command]
```

A completed example is in Section 5.2.

11.13 Sprint: dry_run.md

```
# Sprint [NUMBER] Builder Dry Run
**Date:** [YYYY-MM-DD]
**Builder Tool:** [Tool name]
**Permission Level Requested:** [0-4]

## Files I Intend to Create
| File Path | Change Type | Reason |
|---|---|---|
| src/[path] | Additive | [Why needed] |

## Files I Intend to Modify
| File Path | Change Type | What Changes |
|---|---|---|
| src/[path] | Refactor | [What] |

**STOPPED. Awaiting Architect approval and permission level
assignment.**
```

11.14 Sprint: implementation_log.md

```
# Sprint [NUMBER] Implementation Log
**Date:** [YYYY-MM-DD]
**Builder Tool:** [Tool name]
**Permission Level Used:** [Level]

## Files Created / Modified
- src/[path]: [Description]

## Verification Results

### Test Output
[Paste terminal output here]
Result: Passed / Failed
```

11.15 Sprint: human_review.md

```
# Sprint [NUMBER] Human Approval Gate

## The Five Gates

### Gate 1: Scope
- [ ] implementation_log.md lists only files that appeared in
dry_run.md

### Gate 2: Tests
- [ ] Passing test results are present in implementation_log.md

### Gate 3: Debt
- [ ] All known issues from this sprint are in planning/backlog.md

### Gate 4: Decisions
- [ ] All new dependencies have DECISIONS.md entries

### Gate 5: State
- [ ] STATE.md reflects completed sprint status

## Approved to commit: Yes / No
```

11.16 Sprint: retrospective.md

```
# Sprint [NUMBER] Retrospective

## What Went Well / What Broke or Was Unclear
- [Item]

## Blueprint Quality
- Was requirements.md clear? [Yes / No]
- Was blueprint.md accurate? [Yes / No]

## Time Estimate vs Actual
- Estimated: [X hours] / Actual: [Y hours]

## New Constraints Added to AGENTS.md
- [Any rule added as a result of this sprint's findings]
```

11.17 Sprint: rollback_log.md

```
# Sprint [NUMBER] Rollback Log

## Pre-Implementation (fill in before Builder begins - Level 4
required)
**Current clean commit hash:** [git log --oneline -1]
**Rollback command:** `git checkout [branch] && git revert
HEAD~[N] --no-edit`

## Post-Implementation (fill in only if recovery was needed)
**Root cause identified:** [what went wrong]
**State after revert:** [Clean / Partial - describe]
```

11.18 planning/backlog.md

```
# Backlog

## Backlog Item 001: [Short title]
**Date added:** [YYYY-MM-DD]
**Type:** Feature / Bug / Refactor / Tech Debt
**Priority:** High / Medium / Low / Unprioritized
**Status:** Unprioritized / Accepted / Scheduled for Sprint
[NNN] / Rejected

### Description
[What is this item and why it matters]
```

12. Domain Adaptations and Starter Template Library

12.1 Domain Adaptations

The framework structure is identical across all project types. What changes is the content of the key files — specifically `DOMAIN.md`, `COMMANDS.md`, and `STYLE_GUIDE.md`.

Web Application (Next.js / React)

`DOMAIN.md` entities map to routes, API endpoints, and context components. `COMMANDS.md` manages package scripts such as `npm run build` and `npm run lint`. The Architect treats each route as a domain boundary.

Game Development (Unity / Godot)

`DOMAIN.md` entities track scenes, game states, event handlers, and asset flows. It functions like a technical Game Design Document. `COMMANDS.md` includes build, test, and export commands for the target engine.

Desktop Applications (Electron / Tauri)

Requires explicit local filesystem configuration parameters and safe machine-access bounds tables in `DOMAIN.md`. `SECURITY.md` must explicitly define what local filesystem paths the app is permitted to access.

Data Ingestion & Automation (Python ETL)

Relies heavily on Mode 4 data audits and strict input data validation suites. `DOMAIN.md` defines data shapes, transformation rules, and rejection criteria. `SECURITY.md` must explicitly define which data files may be uploaded to an AI session.

12.2 Starter Template Library

The `_templates/` folder stores pre-built project scaffolds. Copy the closest template, run a find-and-replace on placeholders, fill in business constraints, and initialize your kickoff prompt session.

```
_templates/
├─ web-app/           # Next.js + TypeScript core defaults
├─ game-unity/        # Unity structural asset configurations
├─ game-godot/        # Godot script node structures
├─ desktop-electron/  # Electron main/renderer processes
└─ data-parser/       # Python data handling validation routines
```

```
# Ecosystem setup script sequence
cp -r _templates/web-app/* /path/to/[PROJECT_NAME]/
# Then run global workspace find-and-replace for [PROJECT_NAME].
```

Note: The `_templates/` folder is populated as you build projects. These are not pre-installed templates — they are the starter files you create and refine over time. Your first completed project becomes your first template.

13. Prompt Catalog

How to use these prompts

These prompts are plain text. Copy the prompt, fill in any placeholders in square brackets, and paste it into an AI chat session.

For Architect prompts (13.1, 13.2, 13.3, 13.4, 13.6, 13.8, 13.9): Open a fresh chat window. Paste your project files (`AGENTS.md` , `STATE.md` , `DOMAIN.md` as applicable), then paste the prompt.

For Builder prompts (13.5, 13.7): Open a Builder session in a repository-aware coding tool that has access to your project files. Paste the prompt to invoke the Builder workflow.

If you have no files yet: Start with prompt 13.2 (Project Kickoff Interview). This is the only prompt designed to be used before any files exist. It will ask you questions and generate your foundational files from your answers.

13.1 Persona Injector

Use this at the start of any Architect session to establish the correct mode. Paste it before the rest of your message.

You are the System Architect operating within the Autonomous Duck Deployment Framework for [PROJECT_NAME]. Your role is strictly systems engineering, operational discovery, and strategic orchestration. You operate strictly above the code execution layer.

RIGID CONSTRAINTS:

1. Never write application source files directly.
2. Before approving any work, run a structural Discovery Gate. Interrogate assumptions, surface edge cases, and define clear data verification pipelines.
3. Your primary engineering outputs are structured blueprint packages containing explicit sprint requirements, blueprints, and acceptance parameters.
4. Ground your choices strictly within our DOMAIN.md logic boundaries.

13.2 Project Kickoff Interview Prompt

Use this on your first session, before any other files exist. Paste your raw notes and ideas after this prompt. The AI will ask clarifying questions before producing output.

Execute Project Kickoff Interview Protocol for [PROJECT_NAME].

I will provide raw notes, ideas, rough goals, constraints, and known unknowns. Your task is to ask up to 10 clarifying questions before creating any final architecture. Prioritize questions that affect business logic, data shape, security, user workflows, deployment, and scope boundaries.

After I answer, generate draft contents for:

1. DOMAIN.md
2. STATE.md
3. STYLE_GUIDE.md
4. docs/architecture.md
5. DECISIONS.md starter ledger
6. QUESTIONS.md starter ledger
7. RISKS.md starter ledger

Do not write application source code. Produce Markdown file blocks only.

13.3 Resumption Prompt

Use this when returning to any paused project. Load `STATE.md`, `DOMAIN.md`, and `AGENTS.md` before pasting this prompt.

```
Execute Resumption Protocol.
```

```
Loaded files:
```

- `STATE.md`
- `DOMAIN.md`
- `AGENTS.md`

```
Summarize:
```

1. Current project goal
2. Active sprint or paused status
3. Known blockers
4. Last completed step
5. Next safest action
6. Files that must be loaded before any Builder session

```
Do not invent missing state. If a required detail is absent, add  
it to QUESTIONS.md  
instead of assuming.
```

13.4 Sprint Pack Generation Prompt

Use this in an Architect session to generate a sprint's three files. Load `AGENTS.md`, `STATE.md`, and `DOMAIN.md` before pasting this prompt.

Execute Architect Sprint Generation Protocol.

Active Project State: [Paste current contents of STATE.md]

Target Sprint Scope: Sprint [NUMBER] - [Summarize target feature in one sentence]

Structural References: Refer to our active DOMAIN.md parameters.

Generate a comprehensive blueprint pack for this sprint loop.

Output three distinct

Markdown file blocks ready for local file system saving:

1. REQUIREMENTS.MD: functional business parameters, technical constraints, and clear in-scope versus out-of-scope boundaries.
2. BLUEPRINT.MD: step-by-step implementation design tracking specific files to modify or instantiate with explicit change classifications.
3. ACCEPTANCE.MD: checkbox-format validation requirements mapping clean compilation scripts and automated test coverage rules.

Do not output raw application source code. Provide only blueprint specification parameters.

13.5 Builder Pre-Flight Dry Run Prompt

Use this at the start of every Builder session. The Builder must stop after producing `dry_run.md` and wait for authorization.

Initialize Pre-Flight Dry Run Protocol.

Your current allocated Permission Level for this session: Level 0 (Read-Only).

Active Task Specifications:

- Refer to planning/sprints/sprint_[NUMBER]/requirements.md
- Refer to planning/sprints/sprint_[NUMBER]/blueprint.md

Prior to creating or mutating any file inside the /src directory, execute a read-only analysis of the repository and return a Pre-Flight Summary sheet.

YOUR REQUIRED CHECKLIST:

1. Files to Create: Specify paths and verify intention is Additive.
2. Files to Modify: Specify paths, classify change type (Refactor/Destructive), and provide technical rationale.
3. Out of Scope: Identify files or features explicitly marked out of scope and confirm you will leave them untouched.

CRITICAL INSTRUCTION: Output your report using the standard dry_run.md file format and then STOP immediately. Do not generate source file scripts until you receive explicit operator clearance.

13.6 Architect Sanity Audit Prompt

Use this after the dry run, before authorizing the build. Paste all four sprint files into the session before running this prompt.

```
Execute Architect Sanity Audit Protocol.
```

```
Inputs:
```

- planning/sprints/sprint_[NUMBER]/requirements.md
- planning/sprints/sprint_[NUMBER]/blueprint.md
- planning/sprints/sprint_[NUMBER]/acceptance.md
- planning/sprints/sprint_[NUMBER]/dry_run.md

```
Check the dry run for:
```

1. Hidden scope bleed
2. Files not approved by blueprint.md
3. Business logic not defined in DOMAIN.md
4. Missing verification commands
5. Dependency additions that require DECISIONS.md entries
6. Security or secrets exposure risk

```
Return one of these outcomes:
```

- APPROVED: Permission Level [1-4] authorized, with reason.
- REJECTED: explain required changes to blueprint_feedback.md.

```
Do not write source code.
```

13.7 Builder Implementation Authorization Prompt

Use this in the Builder session after the dry run has been approved by the Architect.

```
Dry run verified by Architect. Permission Level [LEVEL]  
authorized.
```

```
Proceed with implementation according to:
```

- planning/sprints/sprint_[NUMBER]/requirements.md
- planning/sprints/sprint_[NUMBER]/blueprint.md
- planning/sprints/sprint_[NUMBER]/acceptance.md

```
Rules:
```

1. Modify only the approved files.
2. Do not invent new business rules.
3. Log every created or modified file in implementation_log.md.
4. Run all verification commands listed in acceptance.md and COMMANDS.md.
5. Paste raw terminal output into implementation_log.md.
6. Stop if a blocking ambiguity appears.

13.8 Architect Implementation Review Prompt

Use this after the Builder completes implementation, before the human approval gate.

```
Execute Implementation Review Protocol.
```

```
Inputs:
```

- acceptance.md
- dry_run.md
- implementation_log.md
- test output

```
Review whether the implementation satisfies the sprint acceptance criteria. Verify that implementation_log.md lists only files that appeared in dry_run.md unless an approved expansion was granted. Identify any missing tests, unresolved debt, scope creep, or required DECISIONS.md entries.
```

```
Return:
```

1. Approved / Not Approved
2. Required fixes before commit
3. Required updates to STATE.md, DECISIONS.md, RISKS.md, or backlog.md

13.9 Cross-Sprint Consistency Audit Prompt

Use this every third sprint, or whenever you suspect the codebase has drifted from DOMAIN.md.

Execute Cross-Sprint Consistency Audit.

Inputs:

- DOMAIN.md
- STATE.md
- DECISIONS.md
- Current and recent sprint folders
- docs/data_model.md and docs/api.md if applicable

Check for drift between what the blueprints built and what DOMAIN.md says. Identify invented entities, inconsistent field names, outdated docs, unresolved risks, missing decision records, and backlog items that should become sprint candidates.

Return:

1. Consistency score: Green / Yellow / Red
2. Drift findings
3. Required updates before next sprint
4. Recommended next sprint candidates

14. Quick Reference Checklists

These three checklists cover the three most critical moments in the framework. They are short by design — if a checklist takes more than two minutes to complete, it will not be used consistently.

14.1 Before the first AI session

- ☐ `SECURITY.md` has a safe-to-share and never-share list.
 - ☐ `DOMAIN.md` has project overview, core entities, workflows, and out-of-scope boundaries.
 - ☐ `COMMANDS.md` has development, test, build, lint/type-check, deploy, and rollback commands.
 - ☐ `STATE.md` says exactly what step you are on.
 - ☐ `QUESTIONS.md` has no blocker that prevents Sprint 001 planning.
-

14.2 Before Builder writes code

- ☐ Sprint branch exists.
- ☐ `requirements.md`, `blueprint.md`, and `acceptance.md` exist.
- ☐ Builder has completed Level 0 dry run.
- ☐ Architect has approved `dry_run.md` and assigned permission level.
- ☐ New dependencies have `DECISIONS.md` entries.
- ☐

No secrets or PII are loaded into the session.

14.3 Before committing a sprint

- ☐ `implementation_log.md` lists created and modified files.
 - ☐ Test/build/lint output is pasted into `implementation_log.md`.
 - ☐ `human_review.md` passes all five gates: Scope, Tests, Debt, Decisions, State.
 - ☐ `STATE.md` has been updated.
 - ☐ `retrospective.md` records what worked, broke, and should change next time.
 - ☐ Builder thread has been closed after the work is captured in files.
-

15. When Things Go Wrong

The earlier sections describe the happy path. Real projects encounter friction. This section covers the most common failure modes and what to do about each one.

15.1 The Builder touches files outside the blueprint

What happened: You review `implementation_log.md` and find files that were not in `dry_run.md` or `blueprint.md`.

What to do: 1. Do not commit. 2. Revert the unauthorized changes: `git checkout sprint/NNN -- [filename]` 3. Record the violation in `retrospective.md`. 4. Add a constraint to `AGENTS.md`: “Do not modify files outside the approved blueprint scope under any circumstances.” 5. Re-run the sprint from Step 6 (Authorize and Build) with the corrected scope explicitly stated in the authorization message.

15.2 You realize mid-sprint the blueprint is wrong

What happened: While reviewing the Builder’s work, you notice the blueprint is missing a constraint, contains a wrong assumption, or will produce the wrong outcome.

What to do: 1. Stop the Builder session immediately. Do not let it continue against a blueprint you know is wrong. 2. Revert any changes made so far: `git checkout sprint/NNN` 3. Return to the Architect session. Write your corrections in `blueprint_feedback.md`. 4. Run the Sprint Pack Generation Prompt again with `blueprint_feedback.md` loaded. 5. Re-run the dry run against the revised blueprint before authorizing again.

Catching a blueprint error before the full sprint is implemented saves far more time than it costs. This is not a failure — it is the system working as designed.

15.3 You hit a rate limit or token limit mid-session

What happened: The Architect or Builder session is interrupted by a rate limit, token limit, or network error.

What to do: 1. Before closing the interrupted session, save any partial output to the relevant file (`blueprint.md`, `dry_run.md`, etc.). 2. Update `STATE.md` to reflect where you stopped and what was in progress. 3. Open a new session with any available model. Load `AGENTS.md`, `STATE.md`, and the relevant sprint files. 4. Continue from where you stopped.

Because the state is in the files, the new session has full context. The model does not matter. The files do.

15.4 The sprint fails acceptance criteria

What happened: The Builder ran the verification commands and tests are failing, or the build did not complete.

What to do: 1. Do not commit. 2. Record the failure output in `implementation_log.md`. 3. Return to the Builder session. Paste the failing test output and ask for a fix, constrained to the same `blueprint.md` and `acceptance.md`. 4. The Builder must produce a new `dry_run.md` for the fix before implementing it — even if the fix is small. 5. If the failing tests reveal a gap in the blueprint (the Builder cannot fix the failure within the approved scope), return to the Architect to revise the blueprint.

15.5 The Architect's blueprint conflicts with DOMAIN.md

What happened: The Architect generated a blueprint that contradicts a rule in `DOMAIN.md` — for example, it proposes cloud sync for a project where `DOMAIN.md` explicitly says cloud sync is out of scope.

What to do: 1. Do not use the blueprint. 2. In `blueprint_feedback.md`, quote the specific `DOMAIN.md` rule that the blueprint violates. 3. Return to the Architect session and run the Sprint Pack Generation Prompt again with `blueprint_feedback.md` loaded. 4. If the conflict reveals that `DOMAIN.md` is genuinely out of date (you have actually decided to add cloud sync), update `DOMAIN.md` first, create a `DECISIONS.md` entry recording the scope change, then generate a new blueprint.

15.6 You return to a project and STATE.md is out of date

What happened: `STATE.md` was not updated at the close of the last session, or it was updated incompletely.

What to do: 1. Load `DOMAIN.md`, `AGENTS.md`, and the most recent sprint folder. 2. Run the Resumption Prompt (Section 13.3). Ask the Architect to reconstruct the current state from the files that do exist. 3. Use the Architect's reconstruction to manually update `STATE.md`. 4. Add a reminder to the top of `AGENTS.md`: "Always update `STATE.md` at the close of every session. Do not close a session without completing this step."

Prevention: Updating `STATE.md` before closing any session is the single most important habit in this framework. Make it the last action, every time, without exception.

16. Glossary

Term	Meaning
Acceptance Criteria	The checklist in <code>acceptance.md</code> that defines whether a sprint is complete. All items must be checked before the sprint can be committed.
ADR	Architecture Decision Record. A permanent entry in <code>DECISIONS.md</code> recording why a technical decision was made, what was decided, and what the tradeoffs are. Decisions are never deleted — only superseded.
Architect Mode	The planning mode. An LLM invoked in Architect Mode defines requirements, risks, blueprints, and acceptance criteria. It never writes application source code. See Section 9 for full detail.
Builder Mode	The implementation mode. An LLM invoked in Builder Mode writes source code against an approved blueprint. It always runs a dry run first and stops to await permission before writing anything. See Section 9.
Context Bleed	When old or unrelated chat context contaminates the current task. The primary cause of AI regression bugs in vibe coding. Prevented by the state reset at the end of every sprint.
Context Engineering	The practice of organizing project knowledge into clean, specific files rather than relying on long chat history. The core methodology of this framework. See Section 1.3.
Dry Run	A read-only report from the Builder listing every file it intends to create or modify, before any code is written. The Builder stops after the dry run and waits for authorization. See Section 3 (Key Concepts).
Harness-First Development	Writing or defining the test harness before scaling functional implementation. Acceptance criteria are written before code is written.
Local File Sovereignty	The principle that project memory belongs in local Markdown files

Term	Meaning
	controlled by the human operator, not in an AI chat session. See Section 1.
Mode	A task category that defines which workflow to follow: Quick Patch, Feature Sprint, Refactor/Migration, Data Ingestion, or Style Audit. See Section 7.
Permission Level	A number from 0 to 4 that limits what the Builder may do. Always assigned explicitly by the human operator at the start of a Builder session. See Section 9.2.
Sprint Pack	The three files that define a sprint: <code>requirements.md</code> , <code>blueprint.md</code> , and <code>acceptance.md</code> . Generated by the Architect. Consumed by the Builder. See Section 3 (Key Concepts).
Sovereign Brain	The local folder containing the Markdown files that define the project's state, rules, and history.
State Reset	Closing the active Builder chat after the sprint work is captured in files, so the next sprint begins with clean context.
Vibe Coding	An informal AI coding loop where the developer keeps prompting in one chat session without stable project memory, explicit specifications, or a dry run gate. The anti-pattern this framework solves. See Section 1.1.